# LAB7 결과 보고서

*STM32_3 SPI 통신 — MCU와 FPGA 사이 1바이트 송수신 (과제 8~12)*


과목 전자전기컴퓨터설계실험 2

실험일 2026년 11월 2일

성명 김태이

학번 2023440158


GitHub github.com/tae2yi/UOS_ece2/tree/2b72075/lab7

시연 영상 github.com/tae2yi/UOS_ece2/tree/master/lab7/report/final/videos

# 1 실험 범위와 결과 요약

11월 2일 실험에서 NUCLEO-G474RE와 Combo II-DLD 사이에 SPI(Mode 0, 8비트, MSB 먼저, MISO 없음)로 1바이트를 보내는 과제 다섯 개를 했다. 과제 8은 오실로스코프 사진 두 장으로, 과제 9~12는 영상 네 개로 남겼다. 이 보고서는 사진과 영상에서 읽은 값을 예비보고서의 예상과 비교한다. 보드와 화면 사진은 모두 이 사진과 영상의 장면이다.

| 과제 | 확인한 것 | 예상과 비교 |
|---|---|---|
| 8 | SCK 786.2 kHz, 8펄스, 0x55 파형 | 781.25 kHz와 +0.6 % |
| 9 | MCU 수신 RX=00 FF 55 A5 | 순서·값 일치 |
| 10 | FPGA LED FF→55→A5→00, 약 1 s | 일치 |
| 11 | MCU 수신 RX=00 FF 55 A5 반복 | 일치, 타임아웃 없음 |
| 12 | 두 FPGA LED가 같은 값 | 일치 |

다섯 과제 모두 예비보고서가 예상한 값이 나왔다. 예비보고서에서 정하지 못한 CS LOW에서 첫 SCK까지의 시간은 약 3.0 µs로 읽었다. 정량 비교는 오실로스코프 화면 사진의 화소 정밀도(약 ±0.3 µs)와 영상의 프레임 간격(0.033 s) 안에서 한 것이다.

# 2 실험 환경과 측정 방법

| 구분 | 사용한 것 |
|---|---|
| MCU 보드 | NUCLEO-G474RE, 직렬 모니터 9600 8N1 |
| FPGA 보드 | Combo II-DLD, xc7s75fgga484-1, 50 MHz |
| MCU 코드 | ece2_474_routine.c, STM32_3_ROLE 1(송신)·2(수신) |
| FPGA 설계 | spi_rx_uart(수신), spi_tx_uart(송신) |
| 계측기 | Tektronix TBS2000B 디지털 오실로스코프 |
| 기록 | 오실로스코프 사진 2장, 장면 사진 3장, 영상 4개(30 fps) |

배선은 예비보고서의 표를 따랐다. MCU 송신은 D3(PB3, SCK), D11(PA7, MOSI), D10(PB6, CS)에서 나가고, MCU 수신은 같은 D3·D11과 A2(PA4, NSS)로 받는다. FPGA는 BDIO_0(F8)·BDIO_1(E8)·BDIO_2(A7)를 쓰고 DIP SW5(BD_SEL)를 켰다. 모든 연결에서 GND를 함께 이었고, 받는 쪽 프로그램을 먼저 올린 뒤 선을 꽂았다.

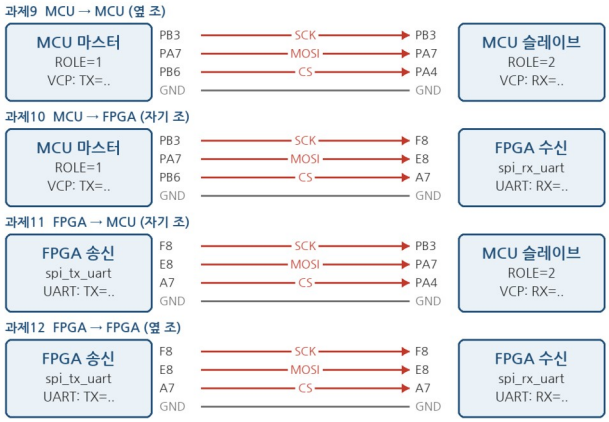



그림 1. 배선(예비보고서 그림). 과제 9와 11에서는 받는 MCU의 CS 입력이 PA4(A2)다.

## 사진과 영상에서 값을 읽은 방법

- 주파수와 전압: 오실로스코프의 자동 측정값을 사진에서 그대로 읽었다.
- 시간 간격: 사진 2에서 신호 에지의 화소 위치를 찾고, SCK 상승 에지 7개 간격(주기 1.272 µs)을 자로 써서 µs로 바꿨다. 화면을 비스듬히 찍어 눈금 간격이 위치마다 조금 달라 약 ±0.3 µs의 오차가 있다.
- 과제 10: 프레임마다 LED1~4 위치의 빨간 화소로 켜짐을 판정해 값이 바뀐 시각을 구했다(한 프레임 0.033 s).
- 직렬 모니터 글자와 과제 12의 LED 값은 장면을 확대해 사람이 읽었다.

측정 코드:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/report/final/make_figures.py

측정값:
https://github.com/tae2yi/UOS_ece2/tree/2b72075/lab7/report/final/data

# 3 과제 8 MCU 송신 파형

## 코드

ROLE=1에서 polling_routine은 1초마다 CS(PB6)를 LOW로 내리고, HAL_SPI_Transmit으로 1바이트를 보낸 뒤 CS를 HIGH로 올린다[1]. SPI1 클록은 APB2 100 MHz를 128로 나눈 781.25 kHz(주기 1.28 µs)다.

```
HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
        SPI_CS_ece2_Pin, GPIO_PIN_RESET);
HAL_StatusTypeDef result =
  HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
        SPI_CS_ece2_Pin, GPIO_PIN_SET);
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/stm32_lab/Core/Src/ece2_474_routine.c#L109-L114

## 측정 장면과 오실로스코프 화면

과제 8 측정 장면
NUCLEO-G474RE 혼자, 프로브 2개를 SCK·MOSI(또는 CS)에 연결

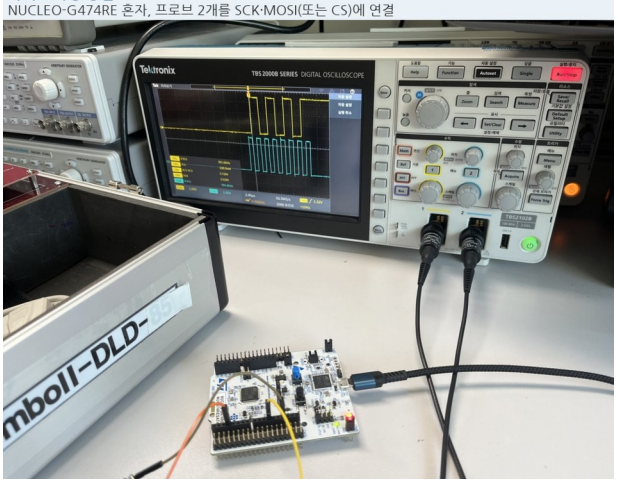



그림 2. 과제 8 측정 장면. 입력 조건: NUCLEO 한 대를 ROLE=1로 실행, 프로브 두 개를 신호와 GND에 연결. 확인한 동작: 1초마다 한 묶음의 파형이 화면에 잡힌다.

사진 1 CH1 MOSI (노랑), CH2 SCK (파랑), 4 us/div
0x55 = 0101_0101, SCK 786.2 kHz, MOSI 391.2 kHz

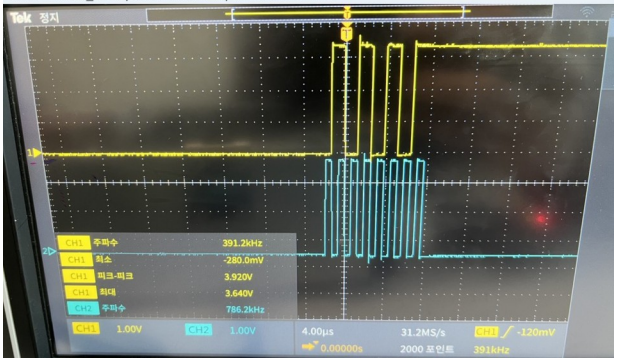



그림 3. 과제 8 오실로스코프 사진 1. 입력 조건: CH1 MOSI(PA7), CH2 SCK(PB3), 4 µs/div, 보낸 값 0x55. 확인한 동작: SCK 8펄스 동안 MOSI가 0, 1을 번갈아 내고, 마지막 비트 1을 그대로 유지한다.

사진 2 CH1 CS (노랑), CH2 SCK (파랑), 4 us/div
CS LOW 동안 SCK 8펄스, CS 상승 에지 트리거

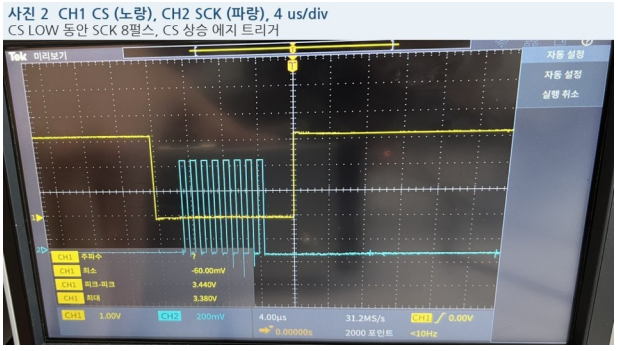



그림 4. 과제 8 오실로스코프 사진 2. 입력 조건: CH1 CS(PB6), CH2 SCK(PB3), 4 µs/div, CS 상승 에지 트리거. 확인한 동작: CS가 LOW인 동안에만 SCK가 8번 나온다.

| 항목 | 값 | 읽은 방법 |
|---|---|---|
| SCK 주파수 | 786.2 kHz | CH2 자동 측정(사진 1) |
| MOSI 주파수 (0x55) | 391.2 kHz | CH1 자동 측정(사진 1) |
| 신호 최대 / 피크-피크 | 3.64 V / 3.92 V | CH1 자동 측정(사진 1) |
| SCK 펄스 수 | 8 | 사진 2에서 셈 |
| CS LOW → 첫 SCK 상승 | 약 3.0 µs | 사진 2 화소 |
| 마지막 SCK 하강 → CS HIGH | 약 3.4 µs | 사진 2 화소 |
| CS LOW 전체 | 약 15.8 µs | 사진 2 화소 |

측정값:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/report/final/data/task8_scope_readout.csv

### 해석

사진 1에서 SCK 상승 에지 때의 MOSI는 0, 1, 0, 1, 0, 1, 0, 1이라 0x55를 MSB부터 보낸 것이다. MOSI는 SCK가 내려갈 때 바뀌고 올라갈 때는 안정되어 있어 Mode 0의 약속과 같다. 0과 1이 번갈아 나오므로 MOSI 주파수는 SCK의 절반이어야 하고, 391.2 kHz는 786.2 kHz의 절반(393.1 kHz)과 0.5 % 안에서 같다.

사진 2에서 CS LOW 약 15.8 µs는 CS → 첫 SCK 3.0 µs, SCK 7.5주기 9.5 µs, 마지막 SCK → CS HIGH 3.4 µs의 합(15.9 µs)과 맞는다. SCK 앞뒤의 3 µs 남짓은 CS를 GPIO로 바꾼 뒤 HAL_SPI_Transmit이 SPI를 설정해 보내기 시작하기까지, 그리고 전송이 끝났는지(BSY) 확인하고 돌아올 때까지 CPU가 쓴 시간이다. 100 MHz에서 약 300클록이다.

### 예상과의 비교

SCK 786.2 kHz는 설계값 781.25 kHz보다 0.6 % 높다. MCU 클록원이 내부 RC 발진기(HSI)라 수정 발진기보다 오차가 크고, 오실로스코프도 40 µs 창을 2000점(32 ns 간격)으로 잡아 한 주기 측정에 몇 %의 분해능 한계가 있다. 그래서 0.6 %를 두 원인으로 나눌 수는 없지만, FPGA 수신은 SCK 에지만 따라가므로 이 정도 차이는 영향이 없다.

FPGA 수신기는 CS가 2클록(40 ns) 동기화를 거친 뒤 첫 SCK 상승 에지를 받아야 한다. 측정한 3.0 µs는 이보다 75배 길어 충분하다. 신호 최대 3.64 V와 최소 −0.28 V는 3.3 V 로직에 프로브 접지선으로 생긴 울림이 더해진 값이다.

# 4 과제 9 MCU에서 MCU로

옆 조와 NUCLEO 두 대를 이어 한쪽은 ROLE=1(송신), 다른 쪽은 ROLE=2(수신)로 실행했다. 수신 쪽은 ece2_474_init에서 SPI1을 슬레이브, NSS 하드웨어 입력(PA4)으로 다시 설정하고, polling_routine에서 HAL_SPI_Receive로 최대 2초 기다린다.

```
  uint8_t rx = 0;
  HAL_StatusTypeDef result =
    HAL_SPI_Receive(&hspi1, &rx, 1U, 2000U);
  if (result == HAL_OK)
  {
    printf("RX=%02X\r\n", (unsigned int)rx);
  }
  else if (result == HAL_TIMEOUT)
  {
    /* 매뉴얼 한 줄을 들로 나눔(같은 문자열) */
    printf("RX timeout: check sender, SCK, MOSI, "
       "CS and GND\r\n");
  }
  else
  {
    printf("RX error=%d\r\n", (int)result);
  }
#endif
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/stm32_lab/Core/Src/ece2_474_routine.c#L120-L137

## 보드 관찰

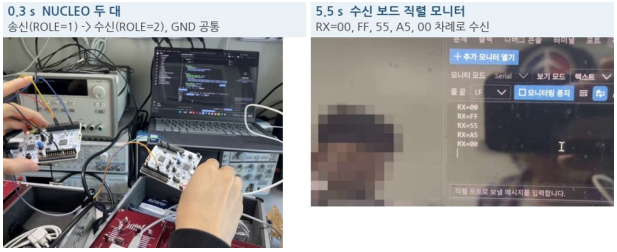



그림 5. 과제 9 장면. 입력 조건: 송신 D3·D11·D10 → 수신 D3·D11·A2, GND 공통. 확인한 동작: 수신 보드 직렬 모니터에 1초마다 받은 값이 한 줄씩 찍힌다.

| 순서 | 화면의 줄 | 보낸 순서와 비교 |
|---|---|---|
| 1 | RX=00 | 첫 줄 |
| 2 | RX=FF | 00 다음 = FF |
| 3 | RX=55 | FF 다음 = 55 |
| 4 | RX=A5 | 55 다음 = A5 |
| 5 | RX=00 | A5 다음 = 00 |

받은 값이 보낸 순서(55 → A5 → 00 → FF → 55 …)를 그대로 따랐고, 타임아웃이나 오류 줄은 영상 구간에 없었다. 1초마다 한 줄씩 늘어나 송신 주기와 같다.

## 예상과의 비교

예비보고서의 호스트 시뮬레이션은 송신 측이 만든 바이트를 수신 측 코드에 넣어 같은 순서가 나오는 것을 확인했고, 보드에서도 같았다. 송신 측 CS(PB6)를 수신 측 NSS(PA4)에 이어 수신 MCU의 하드웨어 NSS가 바이트의 시작과 끝을 알 수 있었다.

# 5 과제 10 MCU에서 FPGA로

## 설계

spi_slave_rx는 SCK·MOSI·CS_n을 2단 플립플롭으로 50 MHz에 맞추고, SCK 상승 에지마다 MOSI를 시프트 레지스터에 넣는다. 8번째 에지에서 바이트를 내고, CS_n이 HIGH가 되면 비트 카운터를 0으로 돌린다[2]. 받은 값은 LED1(bit7)~LED8(bit0)에 보이고 UART로도 "RX=55"처럼 보낸다.

```
      if (cs_n_s) begin
        bit_count <= 0;   // CS 해제: 처음부터
      end else if (sck_rise) begin
        if (bit_count == 3'd7) begin
          data <= {shift, mosi_s};  // 8번째 비트
          valid <= 1;
          bit_count <= 0;
        end else begin
          shift <= {shift[5:0], mosi_s};
          bit_count <= bit_count + 1'b1;
        end
      end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/lab7_01_spi_rx/src/spi_slave_rx.v#L47-L58

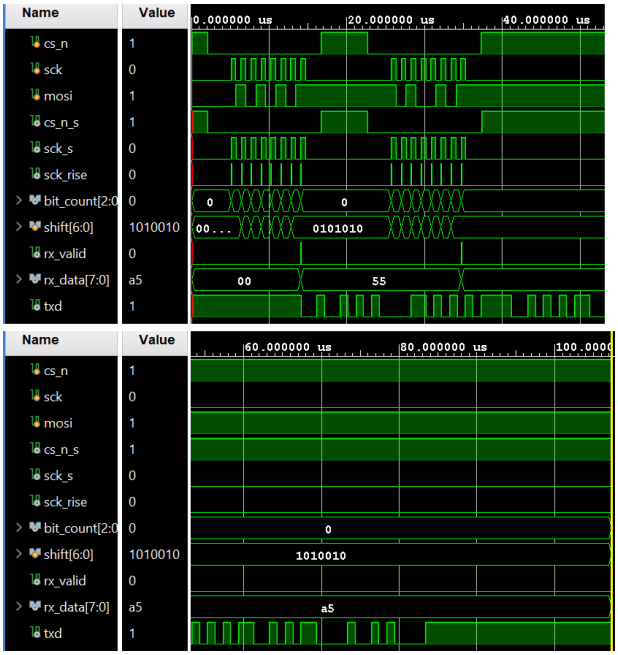



그림 6. 과제 10 FPGA Vivado 파형(예비). MCU 호스트 시뮬레이션의 0x55를 781.25 kHz로 넣으면 8번째 SCK 상승 에지 뒤 rx_data가 55가 된다.

## 보드 관찰

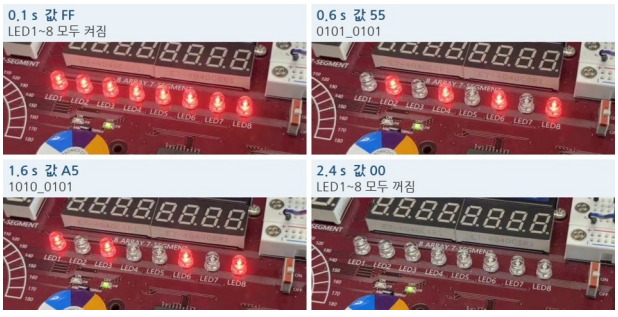



그림 7. 과제 10 보드 사진. 입력 조건: NUCLEO(ROLE=1) D3·D11·D10 → BDIO_0~2, DIP SW5 켬. 확인한 동작: LED1~8이 1초마다 FF → 55 → A5 → 00으로 바뀐다.

| 바뀐 시각 (s) | LED1~4 | LED1~8 (값) | 간격 (s) |
|---|---|---|---|
| 0.20 | 1111 → 0101 | 0101_0101 (55) | — |
| 1.17 | 0101 → 1010 | 1010_0101 (A5) | 0.97 |
| 2.17 | 1010 → 0000 | 0000_0000 (00) | 1.00 |

측정값:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/report/final/data/task10_led1_4.csv

시각은 LED1~4로 구했고, LED5~8은 장면을 보고 읽어 값을 정했다. 손떨림으로 오른쪽 LED가 판정 영역을 벗어나 LED1~4만 자동으로 읽었다. 간격 0.97 s와 1.00 s는 프레임 한 칸(0.033 s) 안에서 1 s와 같다.

### 예상과의 비교

예비보고서의 「LED = MCU가 보낸 값, 1초마다 55·A5·00·FF」와 같다. FPGA 수신이 매번 정확히 8비트를 받았다는 것은 값이 한 비트도 밀리지 않은 것으로 확인된다. FPGA가 UART로 내보낸 "RX=.." 줄은 이번에 USB-시리얼로 보지 않았다.

## 6 과제 11 FPGA에서 MCU로

### 설계

spi_master_tx는 LEAD → HIGH/LOW 8번 → TRAIL → GAP 상태를 반주기(50클록 = 1 µs)마다 넘어가며 SCK 500 kHz를 만든다. CS_n을 먼저 내리고 첫 비트를 MOSI에 놓은 뒤 반주기 뒤에 첫 상승 에지를 낸다. 1초마다 55·A5·00·FF를 차례로 보낸다.

```
        HIGH: if (tick) begin
          sck <= 0; count <= HALF - 1;
          if (bit_index == 3'd7) state <= TRAIL;
          else begin
            shift <= {shift[6:0], 1'b0};
            mosi <= shift[6];  // 다음 비트
            bit_index <= bit_index + 1'b1;
            state <= LOW;
          end
        end
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/lab7_02_spi_tx/src/spi_master_tx.v#L48-L57

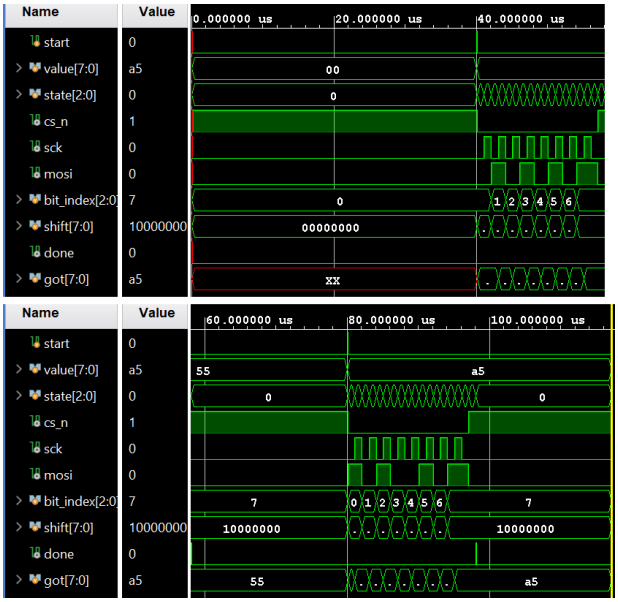



그림 8. 과제 11 FPGA Vivado 파형(예비). CS_n LOW 17 µs 동안 SCK 2 µs 주기로 8펄스, MOSI는 SCK가 LOW일 때만 바뀐다.

### 보드 관찰




그림 9. 과제 11 장면. 입력 조건: FPGA 송신 BDIO_0~2 → NUCLEO(ROLE=2) D3·D11·A2. 확인한 동작: MCU 직렬 모니터에 RX=00, FF, 55, A5가 순서대로 반복된다.

2.5 s 화면에 보이는 15줄이 RX=00, FF, 55, A5의 반복으로, FPGA가 보내는 순서를 세 바퀴 넘게 그대로 따랐다. 수신 MCU의 SPI 클록은 FPGA에서 오므로 10번(781 kHz)과 주기가 달라도(500 kHz) 같은 설정으로 받았다. 영상 11.5 s 동안 타임아웃 줄은 없었다.

### 예상과의 비교

예비보고서에서 FPGA 테스트벤치 출력을 MCU 호스트 시뮬레이션의 수신에 넣어 본 결과와 같은 순서다. FPGA 송신의 SCK 2 µs와 CS LOW 17 µs는 오실로스코프로 재지 않아 시뮬레이션 값으로만 남았다. 예비보고서에서 걱정한 「수신 MCU가 바이트 중간부터 받아 값이 밀림」은 기록한 구간에서 나오지 않았다.

# 7 과제 12 FPGA에서 FPGA로

옆 조 보드와 한 대에는 spi_tx_uart, 다른 한 대에는 spi_rx_uart를 올리고 BDIO_0~2를 같은 번호끼리, GND를 함께 이었다. MCU는 쓰지 않았다. 예비보고서에서는 두 보드의 클록을 20.000 ns와 20.050 ns로 다르게 두고 선 지연을 넣은 시뮬레이션으로 같은 값을 받는 것을 확인했다.

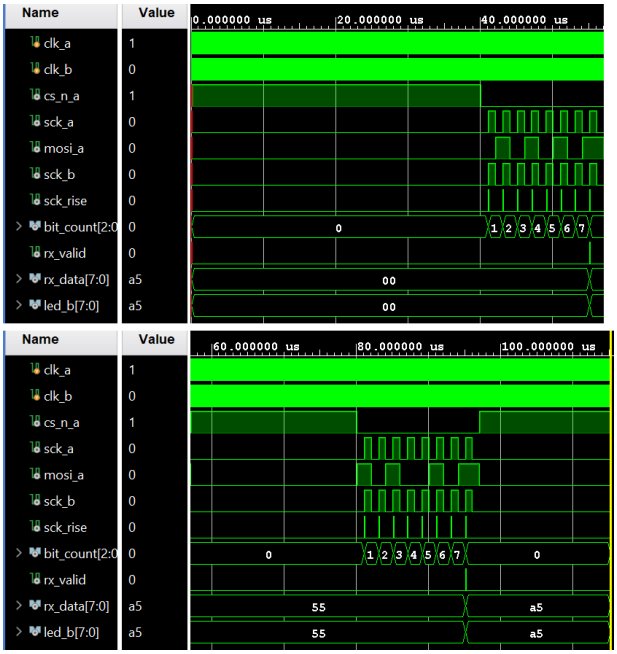



그림 10. 과제 12 Vivado 파형(예비). 클록이 다른 두 보드 사이에서도 수신 보드의 rx_data가 보낸 값과 같다.

## 보드 관찰




그림 11. 과제 12 장면. 위가 송신 보드, 아래가 수신 보드. 입력 조건: 송신 BDIO_0~2 → 수신 BDIO_0~2, GND 공통, 두 보드 모두 DIP SW5 켬.

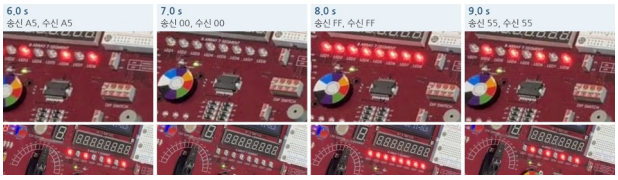



그림 12. 과제 12 두 보드의 LED. 각 칸의 위는 송신 보드, 아래는 수신 보드의 LED1~8. 확인한 동작: 수신 보드가 같은 시각에 송신 보드와 같은 값을 보인다.

| 시각 (s) | 송신 보드 | 수신 보드 | 같은가 |
|---|---|---|---|
| 6.0 | A5 | A5 | 같음 |
| 6.5 | 00 | 00 | 같음 |
| 7.0 | 00 | 00 | 같음 |
| 7.5 | FF | FF | 같음 |
| 8.0 | FF | FF | 같음 |
| 8.5 | 55 | 55 | 같음 |
| 9.0 | 55 | 55 | 같음 |

측정값: https://github.com/tae2yi/UOS_ece2/blob/2b72075/lab7/report/final/data/task12_led_readout.csv

영상의 위쪽 보드가 송신, 아래쪽 보드가 수신이다. 송신 보드의 LED가 A5 → 00 → FF → 55로 바뀔 때 수신 보드의 LED도 같은 순간에 같은 값으로 바뀌었다. 수신 보드의 LED는 받은 바이트이므로, 보낸 값이 그대로 도착했다.

## 예상과의 비교

예비보고서의 「수신 보드 LED = 송신 보드 LED」와 같다. 두 보드의 50 MHz는 서로 다른 발진기에서 나오지만, 수신기는 SCK를 같이 받아 그 에지에 맞춰 읽으므로 두 클록이 다른 것이 문제가 되지 않았다. UART와 달리 받는 쪽이 속도를 미리 알 필요가 없다.

# 8 토론

## 예상과 실제의 비교

| 예상 (예비보고서) | 실제 | 판정 |
|---|---|---|
| MCU SCK 781.25 kHz | 786.2 kHz | +0.6 % |
| 한 바이트에 SCK 8펄스 | 8펄스 | 일치 |
| 0x55 MSB 먼저, Mode 0 | 0101_0101, 하강 에지에서 바뀜 | 일치 |
| CS → 첫 SCK (HAL 시간) | 약 3.0 µs | 새로 측정 |
| 과제9 MCU 수신 = 송신 값 | RX=00 FF 55 A5 00 | 일치 |
| 과제10 LED = 보낸 값, 1 s | FF 55 A5 00, 0.97·1.00 s | 일치 |
| 과제11 MCU 수신 = FPGA 값 | RX=00 FF 55 A5 반복 | 일치 |
| 과제12 두 보드 LED 같음 | 7장면 모두 같음 | 일치 |
| FPGA SCK 2 µs, CS 17 µs | 오실로스코프 없음 | 미확인 |

### 클록 선이 있는 직렬 통신

LAB6의 UART는 양쪽이 9600 bps를 따로 만들어 속도 차이가 받는 쪽 여유를 깎았다. SPI는 보내는 쪽이 SCK를 함께 보내므로, 이번에 MCU 수신이 781 kHz(과제 9)와 500 kHz(과제 11)를 같은 코드로 받았고, FPGA 수신도 MCU와 다른 FPGA를 같은 bitstream으로 받았다. 대신 선이 세 개(SCK, MOSI, CS)와 GND로 늘었고, 받는 쪽이 잘 받았는지 보내는 쪽은 알 수 없다. MCU 송신의 result=0은 보냈다는 뜻일 뿐이라, 받은 값은 받는 쪽 LED와 직렬 모니터로 확인했다.

### 오차와 한계

- 오실로스코프 시간 간격은 비스듬히 찍은 화면 사진에서 읽어 약 ±0.3 µs의 오차가 있다. 화면 저장(USB) 기능을 쓰지 않았다.
- 영상의 시간 분해능은 0.033 s라 1초 주기 확인까지만 할 수 있다.
- FPGA 송신 파형과 FPGA의 UART 출력은 계측하지 않았다.

### 개선 방법

- 오실로스코프의 Save 기능으로 파형 CSV를 저장해 CS → SCK 시간과 주기를 커서로 정확히 잰다.
- 과제 11에서 FPGA 송신 SCK·CS를 함께 찍어 2 µs, 17 µs를 확인한다.
- 수신 MCU를 송신 도중에 리셋해 값이 밀리는지, CS로 다시 맞춰지는지 시험한다.

## 9 결론

STM32 SPI1 송신은 SCK 786.2 kHz(설계 781.25 kHz, +0.6 %)로 CS LOW 동안 8펄스를 내고, 0x55를 MSB부터 Mode 0으로 보냈다. CS를 내린 뒤 첫 SCK까지 약 3.0 µs, 마지막 SCK 뒤 CS를 올리기까지 약 3.4 µs가 HAL 처리 시간으로 들어갔다.

MCU → MCU, MCU → FPGA, FPGA → MCU, FPGA → FPGA 네 조합 모두 1초마다 55·A5·00·FF가 순서와 값 그대로 전달되었다. 직접 설계한 FPGA SPI 수신기와 송신기가 MCU와도, 클록이 다른 FPGA와도 맞물려 동작했다.

## 10 제출 자료

| 항목 | 위치 |
|---|---|
| 저장소 | https://github.com/tae2yi/UOS_ece2 |
| 커밋 | 2b72075b237e335f139344296c7f057ead3dddc6 |
| MCU 코드 | lab7/stm32_lab/Core/Src/ece2_474_routine.c |
| FPGA 설계 | lab7/lab7_01_spi_rx, lab7_02_spi_tx |
| 영상·설명 | lab7/report/final/videos/README.md |
| 오실로스코프 | lab7/report/final/images/scope/ |
| 측정값 | lab7/report/final/data/*.csv |

소스: https://github.com/tae2yi/UOS_ece2/tree/2b72075/lab7

영상: https://github.com/tae2yi/UOS_ece2/tree/master/lab7/report/final/videos

## 참고 문헌


[1] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal_spi.c). https://github.com/STMicroelectronics/STM32CubeG4

[2] Sarah L. Harris and David Harris, Digital Design and Computer Architecture: ARM Edition, Morgan Kaufmann, 2015. ISBN 978-0-12-800056-4.